CPSC 250
Branching and Flow Control

Edward Brash

1 Introduction

One of the most important ideas in programming is the ability for a program to make decisions.
Programs are not simply sequences of instructions executed line-by-line forever.
Instead, programs often need to:

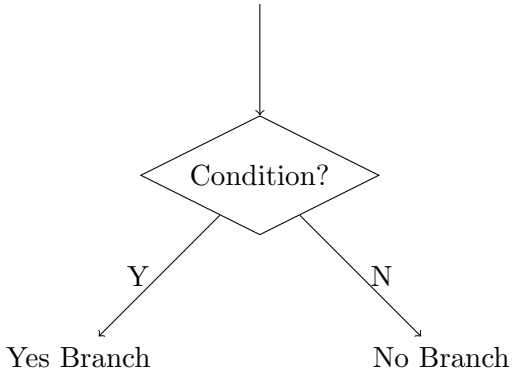
- make decisions,
- repeat actions,
- and alter execution flow dynamically.

This is called:

flow control

2 Branching

Branching occurs whenever a program must decide between multiple execution paths.
Conceptually:



3 The if-else Structure

The most general branching structure in Python is:

```
if condition:
    # do something
else:
    # do something else
```

Sometimes the `else` branch simply does nothing.
In that case:

```
if condition:
    # do something
```

is really just shorthand for:

```
if condition:
    # do something
else:
    pass
```

4 Conditions Evaluate to Booleans

A condition must evaluate to a Boolean value:

`True`

or

`False`

Example:

```
a == 4
```

This expression evaluates to:

- `True` if `a` equals 4
- `False` otherwise

5 Comparison vs Assignment

A very common mistake for beginners is confusing:

```
==
```

with:

```
=
```

These are completely different operations.

5.1 Logical Comparison

```
a == 4
```

asks:

"Is the current value of `a` equal to 4?"

5.2 Assignment

```
a = 4
```

means:

"Store the value 4 into the memory location associated with `a`."

6 Loops

Loops allow a section of code to execute repeatedly.

There are two major loop structures in Python:

1. `for` loops
2. `while` loops

7 For Loops

A `for` loop is useful when you know approximately how many times you want to execute a loop.

Example:

```
for i in range(10):
    print(i)
```

This loop executes 10 times:

$$i = 0, 1, 2, \ldots, 9$$

7.1 Looping Through Collections

```
l = ['a', 'b', 'c']

for item in l:
    print(item)
```

Output:

```
a
b
c
```

8 While Loops

A `while` loop is useful when you do *not* know in advance how many iterations are required.

General structure:

```
while condition:
    # do stuff
```

The loop continues executing while the condition remains true.

Example:

```
a = 0

while a < 4:
    print(a)
    a += 1
```

The loop terminates once the condition becomes false.

9 Infinite Loops

One danger with `while` loops is accidentally creating infinite loops.

Example:

```
while True:
    print("Oops")
```

This loop never terminates unless interrupted externally.

10 Break Statements

Sometimes loops contain multiple conditions for termination.

Example problem:

> Loop through a list until reaching the value `"end"`.

```
l = ["bob", "alice", "fred", "end", "jane"]

for item in l:

    if item == "end":
        break

    print(item)
```

Output:

```
bob
alice
fred
```

The `break` statement immediately exits the loop.

11 Key Takeaways

- Branching controls program decision-making.
- Conditions evaluate to Boolean values.
- `==` performs comparison.
- `=` performs assignment.
- `for` loops are ideal for known iteration counts.
- `while` loops are ideal for condition-based iteration.
- `break` immediately exits a loop.